305432 เครือข่ายเซนเซอร์ (Sensor Networks)

Publish-Time Optimizations for Web-Based Visualizations

การเตรียมงานตอนเผยแพร่เพื่อเพิ่มประสิทธิภาพเว็บแสดงภาพข้อมูล

Ron Pechuk, Jeffrey Heer
University of Washington

2025 IEEE Visualization and Visual Analytics (VIS), pp. 226–230

IEEE Doc ID: 11298742 DOI: 10.1109/VIS60296.2025.00051

นำเสนอโดย นายธีรภัทร ภู่ระย้า รหัสนิสิต 66362416

ปัญหาและตัวชี้วัดของการเปิดเว็บ

งานที่ผู้ใช้ครั้งแรกยังต้องรอ

1 **โหลดและคำนวณข้อมูล**

คิวรีเริ่มต้นและการวาดภาพเกิดหลังเปิดหน้า

2 **สร้างแคชตอนใช้งาน**

ผู้ใช้ครั้งแรกยังต้องสร้างผลที่คนอื่นใช้ซ้ำ

3 **ภาพกับระบบโต้ตอบพร้อมต่างเวลา**

ภาพเริ่มต้นอาจมาแล้ว แต่ยังรับอินพุตไม่ได้

4 **ทรัพยากรล่วงหน้ามีต้นทุน**

ต้องแลกพื้นที่และจังหวะสร้างข้อมูลใหม่

TTR เวลาเห็นภาพ

จากร้องขอหน้า จนเห็น visualization ครบ

นับเนื้อหาภาพที่มีความหมาย ไม่ใช่เพียง header

TTA เวลาพร้อมโต้ตอบ

จนระบบตอบ filter, zoom หรือ selection ได้

วัดความพร้อมเริ่มใช้งาน แยกจากเวลาเห็นภาพ

SC พื้นที่ทรัพยากรที่เผยแพร่

พิจารณาพื้นที่ร่วมกับผลต่อการโหลดและหน่วยความจำ

ที่มา: Section 3.1, หน้า PDF 2

สถาปัตยกรรม Mosaic Publisher

กลยุทธ์ก่อน user request

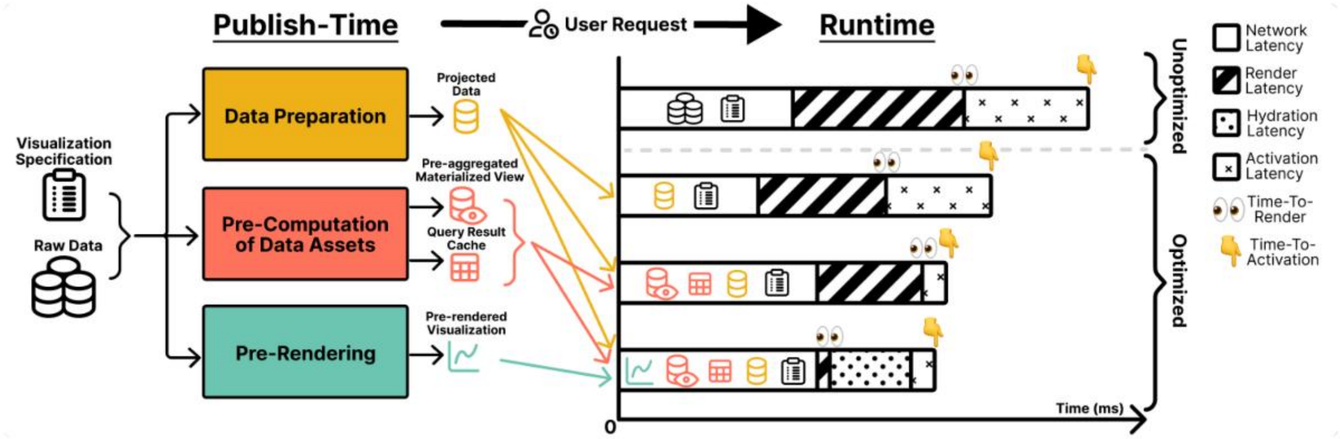


Figure 1: งานสามกลุ่มที่ย้ายไปช่วงเผยแพร่ (หน้า PDF 1)

กระบวนการของ Publisher

จำลองอินพุต virtual DOM → **คำนวณและเก็บผล DuckDB** → **rewrite AST สร้าง assets และ JS**

สรุปกลไกจาก Sections 4.1–4.3, หน้า PDF 3

ทรัพยากรที่ส่งให้ runtime

1 ข้อมูลที่เตรียมไว้

เลือกคอลัมน์และแปลงข้อมูล
ลดงานเตรียมตอนเปิดหน้า

2 ผลคิวรีและ aggregates

เติม cache และ database
ผ่าน coordinator ของ Mosaic

3 ภาพเริ่มต้นและ JavaScript

แสดงภาพก่อน
แล้วเริ่มคอมโพเนนต์โต้ตอบ

Pre-rendering และต้นทุน Hydration

งานที่ยังเหลือเมื่อมีภาพล่วงหน้า

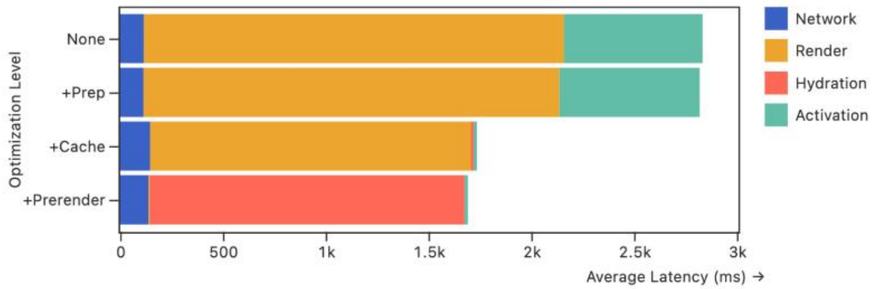


Figure 4: ค่าเฉลี่ย 5 specifications ที่ข้อมูล 10M แถว (หน้า PDF 4)

ภาพเริ่มต้นกับระบบโต้ตอบ

เห็นภาพเริ่มต้น
TTR

→

เริ่มคอมโพเนนต์
Hydration

→

พร้อมรับอินพุต
TTA

ผังสรุปจาก Section 4.3 แสดงเหตุการณ์ ไม่ใช่เวลา benchmark

รูปแบบ view และข้อจำกัด

1 View ที่ไม่โต้ตอบ

ส่งเฉพาะเนื้อหาที่เรนเดอร์ไว้
ไม่ต้องเพิ่มตรรกะ runtime

2 View ที่โต้ตอบ

ส่ง initial view พร้อม JavaScript
เติมแคชและเริ่มส่วนโต้ตอบ

3 Hydration ยังมีต้นทุนสูง

เกือบเท่า full client-side render
วิธีที่ละเอียดขึ้นเป็น future work

ที่มา: Sections 4.3 และ 5, หน้า PDF 3–4

ผลการทดลองและการแลกพื้นที่กับความหน่วง

TTR และ TTA ตามขนาดข้อมูล

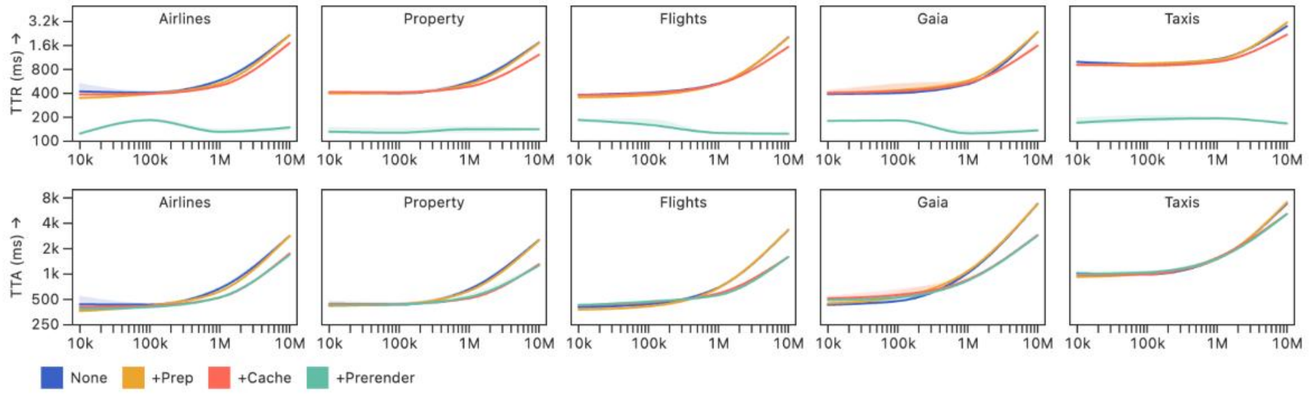


Figure 3: เส้น median, แรเงา IQR, ข้อมูล 10k–10M แถว (หน้า PDF 4)

ค่าเฉลี่ยเปลี่ยนแปลงเทียบ none

ระดับ	ΔSC %	ΔTTR %	ΔTTA %
+prep	0.0 ± 0.0	−0.6 ± 4.0	−1.0 ± 2.5
+cache	21.1 ± 29.8	−16.6 ± 1.3	−32.3 ± 8.9
+most	21.2 ± 29.9	−83.7 ± 2.1	−33.3 ± 9.6

Table 1: mean ± σ ข้ามรูปแบบและขนาดข้อมูล

แถว +most คือชุดรวม
กราฟและข้อความใช้ชื่อ +prerender

−83.7%

เวลาเห็นภาพ TTR

σ = 2.1 (Table 1, ชุดรวม)

−33.3%

เวลาพร้อมโต้ตอบ TTA

σ = 9.6 (Table 1, ชุดรวม)

+21.2%

พื้นที่เผยแพร่ SC

σ = 29.9 (Table 1, ชุดรวม)

5 specifications M1 Pro / RAM 16 GB Node 22.9.0 local server / DuckDB-WASM

แคชลด TTA แล้ว 32.3% ผลชุดรวมเพิ่มการลดอีก 1.0 จุดเปอร์เซ็นต์จาก baseline เดียวกัน

คุณูปการและข้อจำกัดของบทความ

คุณูปการและจุดแข็ง

1 มี baseline และเพิ่มกลยุทธ์เป็นขั้น

แยกผลของงานเตรียม แคช และภาพล่วงหน้า
โดยคงวิธีเขียน specification เดิม

2 ประเมินการโต้ตอบหลายรูปแบบ

5 specifications, ข้อมูล 10k–10M แถว
ทั้ง view เดียวและ dashboard หลาย view

3 วัดทั้งผลและต้นทุน

TTR กับ TTA แสดงความพร้อมคนละช่วง
SC ทำให้เห็นพื้นที่ที่ต้องแลก

ที่มา: Sections 3–6 และ Table 1, หน้า PDF 2–4

ข้อจำกัด

1 Hydration ยังทำงานมาก

สร้างคอมโพเนนต์แบบ client-side rendering
ยังไม่ทดลอง progressive hydration

2 ทรัพยากรเก่าและพื้นที่เพิ่ม

ข้อมูลเปลี่ยนต้องสร้างใหม่
SC +21.2% มี σ สูงถึง 29.9

3 ขอบเขตการทดลองยังจำกัด

laptop เครื่องเดียวและ local server
ไม่แจกแจงจำนวนรอบทำซ้ำ

ข้อสรุปและแนวทางวิจัยต่อ

ข้อสรุปจากการอ่าน

1 แคชลดงานก่อนเปิดการโต้ตอบ

+cache ลด TTA เฉลี่ย 32.3%
เทียบกับ none

2 ภาพล่วงหน้าลดเวลารอเห็นภาพ

ชุดรวมลด TTR 83.7% แต่ TTA 33.3%
ยังเหลือต้นทุน Hydration

3 พื้นที่ทรัพยากรเพิ่มขึ้น

SC เพิ่ม 21.2% ± 29.9
พื้นที่เพิ่มต่างกันระหว่างเงื่อนไข

ข้อสรุป: Figure 4 / Table 1 แนวทางผู้วิจัย: Sections 5–6, หน้า PDF 4

แนวทางที่ทีมวิจัยเสนอ

1 ลดต้นทุน Hydration

ศึกษาวิธี hydration ที่ละเอียดขึ้น
เพื่อลดงานเริ่มส่วนโต้ตอบ

2 ผสาน Publish time กับ Runtime

Hybrid strategies และ incremental regeneration
เป็นแนวทางขยายวิธีปรับปรุง

3 เลือกการปรับปรุงโดยอัตโนมัติ

พัฒนาวิธีเลือก optimization configuration
จากข้อมูลและข้อกำหนดภาพ